Software Requirements Specification

Project: Skill/Interest-Based Secure Matching Platform for Erasmus and Local Students

Course: BIL 482 Design Patterns – TOBB University of Economics and Technology, Fall 2026

Instructor: Dr. Çiğdem Avcı

Assignment: Project Assignment 1 (PA1)

Team: Sangre y Oro – Alberto Alvarez Polo (261118082), Alejandro Blas Piris (261118081), Carlos Molina Perez (261118083)

Document authors: Carlos Molina Perez (main author and integration); Alberto Alvarez Polo (Section 6, Software Architecture Overview); Alejandro Blas Piris (Section 7, Pattern–Use Case Mapping)

Table of Contents

1. Introduction . . . 3
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Definitions and Traceability Conventions . . . 3

2. Overall Description . . . 3
2.1 Product Perspective . . . 3
2.2 Product Functions . . . 4
2.3 User Characteristics . . . 4
2.4 Constraints . . . 4
2.5 Assumptions and Dependencies . . . 5

3. System Features (Use Case Based) . . . 5
3.1 Use Case Diagram . . . 6
3.2 Use Case 1: Register & Create Profile (UC1) . . . 6
3.3 Use Case 2: Find a Match (UC2) . . . 8
3.4 Use Case 3: Create/Join Event & Get Notified (UC3) . . . 9
3.5 Use Case 4: Report Inappropriate Content (UC4) . . . 10

4. Non-Functional Requirements . . . 12
4.1 Usability . . . 12
4.2 Performance . . . 12
4.3 Portability . . . 12
4.4 Reliability . . . 12
4.5 Security and Privacy . . . 13
4.6 Maintainability . . . 13

5. External Interface Requirements . . . 13
5.1 User Interfaces . . . 13
5.2 Software Interfaces . . . 14

6. Software Architecture Overview . . . 14
6.1 Architectural Style: Layered Architecture . . . 14
6.2 Architecture Diagram . . . 15
6.3 Layer Descriptions . . . 15
6.4 How the Architecture Supports the Use Cases . . . 15

7. Pattern–Use Case Mapping . . . 16
7.1 Mapping Matrix and Justification . . . 16
7.2 Pattern Participants in the Architecture . . . 17

8. Traceability Matrix . . . 17

9. Task Matrix . . . 18

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes what the Erasmus & Local Student Matching Platform must do and the qualities it must have. It defines the four core use cases (UC1–UC4), the non-functional requirements and the external interfaces, and it brings together the Software Architecture Overview and the Pattern–Use Case Mapping produced in PA1. It is the reference document for the design (PA2, PA3), the implementation and the acceptance tests of the project.

1.2 Scope

The platform is a multilingual web application (Turkish, English, Spanish) that matches Erasmus and local students by shared skills, hobbies and language goals, and lets them create and join events. Native mobile apps, real-time chat, payments and real production integration with eduroam and a commercial translation API are out of scope (see the Project Definition Document, Section 4).

1.3 Definitions and Traceability Conventions

Term / ID	Meaning
Erasmus student	Incoming exchange student hosted by the university for one or two semesters.
Local student	Regular student of the host university.
Match	A confirmed connection between two students, created when a match request is accepted.
UCn	Core use case n (UC1–UC4). Steps are numbered UCn-S1, UCn-S2…; alternative flows UCn-A1…; exceptions UCn-E1…
FR-n.m	Functional requirement m derived from use case n.
NFR-X.n	Non-functional requirement (U = usability, P = performance, PO = portability, R = reliability, S = security & privacy, M = maintainability).
AT-UCn-m	Acceptance test m for use case n (see Section 8).

These identifiers are kept unchanged in the design documents, the source code (class and package names, commit messages) and the tests, so every requirement can be traced from this document to its implementation and its test.

2. Overall Description

2.1 Product Perspective

The platform is a new, stand-alone web application developed as the group course project of BIL 482 Design Patterns. It is not part of an existing university system, but it is designed to sit next to two external systems that it reaches only through Adapters:

- **eduroam authentication** – used to verify that a user belongs to an academic institution (simulated with a mock implementation in this project).

- **Translation API** – used to translate user-written content such as profile descriptions and event texts (simulated with a mock implementation).

Internally the system follows a Layered Architecture (Presentation, Business and Data Access layers), described in Section 6. The design is built so that the mock services can later be replaced by the real ones without changes to the business logic.

2.2 Product Functions

ID	Function	Use case
PF-1	Register through eduroam or an invitation code and create a profile with skills, hobbies, languages and matching preferences.	UC1
PF-2	Find compatible students with a selectable matching criterion (language exchange, hobbies, sports) and personal filters, and send match requests.	UC2
PF-3	Create, browse, join and leave events and activities.	UC3
PF-4	Notify users about new matches, match requests and changes in the events they follow.	UC2, UC3
PF-5	Report inappropriate content and handle it through automatic filter, moderator and administrator.	UC4
PF-6	Show the whole interface in Turkish, English or Spanish and translate user content on request.	All

2.3 User Characteristics

User class	Description	Expected skill level
Erasmus student	International exchange student, usually 19–28 years old, staying one or two semesters. Often has little or no Turkish and wants to meet local students and practise languages.	High digital literacy; uses web apps daily. May not read Turkish, so the interface must work fully in English or Spanish.
Local student	Student of the host university who wants to practise foreign languages, take part in cultural exchange or find partners for hobbies and sports.	High digital literacy. No training expected; the system must be usable on first visit.
Moderator	University staff member or student volunteer who reviews reports that the automatic filter cannot resolve.	Medium technical skill. Needs a short introduction (under 30 minutes) to the moderation panel.
Administrator	Staff member responsible for the platform; handles escalated reports and account suspensions and manages invitation codes.	Good technical skill; familiar with web administration tools.

2.4 Constraints

- **Platform:** web application accessed through a standard web browser; no native mobile app.
- **Architecture:** must follow the Layered Architecture in Section 6, and each use case must apply at least one design pattern (course requirement).
- **Languages and technology:** web stack (HTML/CSS/JavaScript) for the Presentation Layer, an object-oriented backend language for the Business Layer and a relational database for the Data Access Layer (Project Plan, Section 5). The final technology choice is confirmed in PA3.

- **External services:** eduroam and the translation API are not integrated in production; they are simulated with mock implementations behind Adapters.
- **Interface languages:** Turkish, English and Spanish are mandatory.
- **Schedule:** the work must fit the Fall 2026 semester: PA1 on 26 Oct 2026, PA2 workshop on 16 Nov 2026, PA3 on 7 Dec 2026, code freeze on 30 Dec 2026 and final demo on 4 Jan 2027 (planned dates, Project Plan Section 4).
- **Team and tools:** three developers; all work is versioned on GitHub, with each member committing from an individual account.
- **Budget:** no paid services, licences or hosting are used.
- **Legal:** GDPR/KVKK principles (data minimisation, consent, right to delete) are followed as design requirements, without formal certification.

2.5 Assumptions and Dependencies

- Users have an Internet connection and an up-to-date browser.
- Erasmus students either have eduroam credentials from their institution or receive an invitation code from the international office.
- The university provides at least one moderator and one administrator during the pilot.
- The mock eduroam and translation services expose the same operations a real service would, so they can be swapped through the Adapters.

3. System Features (Use Case Based)

This section breaks the system down into the four core use cases. Each use case lists the main flow, the alternative flows and the exceptions, followed by the functional requirements derived from it.

3.1 Use Case Diagram

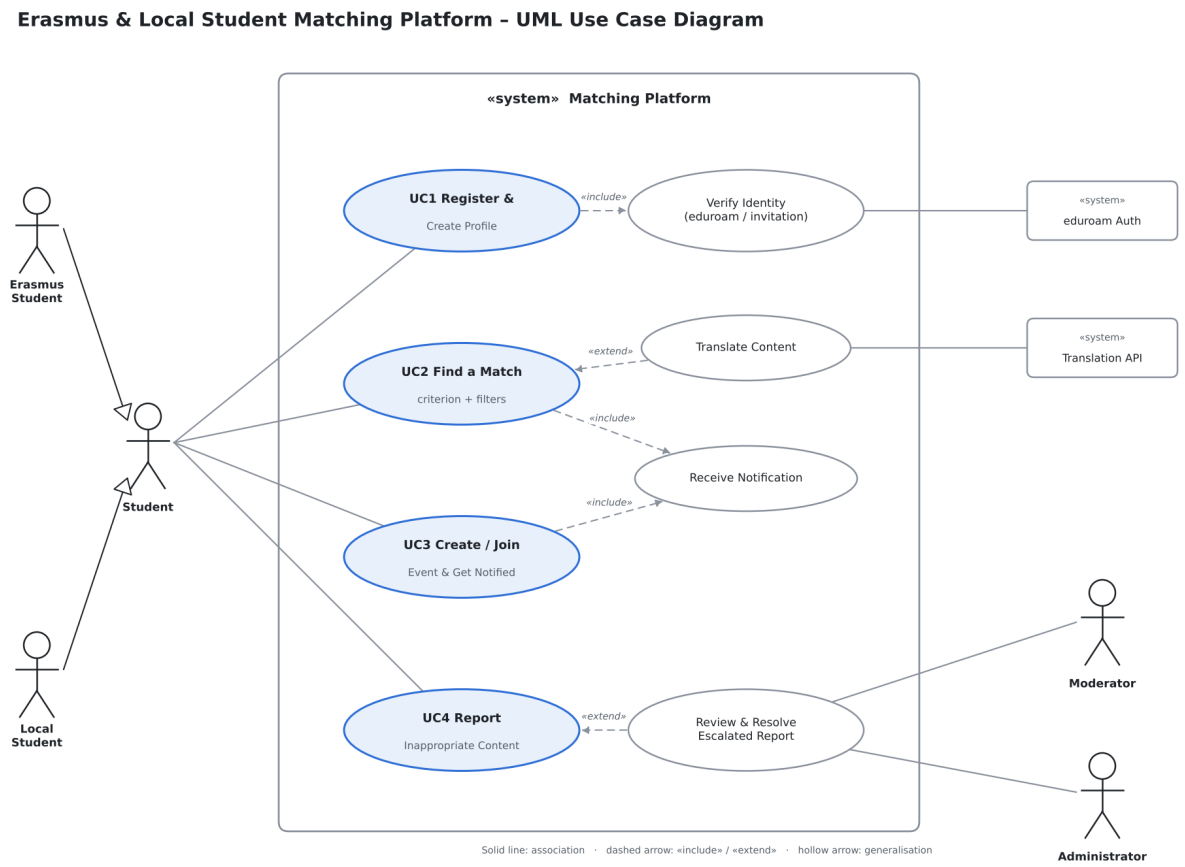


Figure 1 – UML use case diagram of the platform

Actors. *Student* is the primary actor of the four core use cases; *Erasmus Student* and *Local Student* are specialisations of it. *Moderator* and *Administrator* take part in UC4 when a report is escalated. *eduroam Auth* and *Translation API* are external systems reached through Adapters. *Verify Identity* is always included in UC1; *Receive Notification* is included in UC2 and UC3; *Translate Content* extends UC2 when the profile language differs from the user's language; and *Review & Resolve Escalated Report* extends UC4 when the automatic filter cannot resolve the report.

3.2 Use Case 1: Register & Create Profile (UC1)

ID / Title	UC1 – Register & Create Profile
Main actor	Student (Erasmus or local)
Secondary actors	eduroam Authentication Service (through EduroamAdapter)
Goal	Create a verified account and a profile with skills, hobbies, languages and matching preferences, so the student can be matched with others.
Preconditions	1. The student does not have an account on the platform. 2. The student has eduroam credentials or a valid, unused invitation code. 3. The platform is online.
Trigger	The student selects *Register* on the landing page.

Main flow	**UC1-S1** The student opens the platform and chooses the interface language (Turkish, English or Spanish). **UC1-S2** The student selects *Register* and chooses *Sign in with eduroam*. **UC1-S3** The student enters institutional eduroam credentials. **UC1-S4** UserService sends the credentials to the AuthenticationProvider; the EduroamAdapter translates the call to the eduroam service and returns the verified identity (name, e-mail, institution). **UC1-S5** The system asks the student to choose a student type (Erasmus or local) and to complete basic data (home university, host university, department and, for Erasmus students, exchange period). **UC1-S6** UserService asks the corresponding creator (ErasmusStudentCreator or LocalStudentCreator, Factory Method) to build the user object. **UC1-S7** The student fills in the profile: skills, hobbies, native language(s), languages to practise with level (A1–C2), preferred matching criteria, short description and privacy settings. **UC1-S8** The system validates the profile (required fields, at least one hobby or skill and at least one language to practise). **UC1-S9** UserService stores the user and the profile through UserRepository in a single transaction. **UC1-S10** The system logs the student in, shows a confirmation message and offers to start the first search (UC2).
Alternative flows	**UC1-A1 Invitation code.** At UC1-S2 the student chooses *Register with invitation code*, enters the code, and the system checks that it exists, is unused and has not expired. The flow continues at UC1-S5 and the student also enters an e-mail address and a password. **UC1-A2 Save draft.** At UC1-S7 the student can save an incomplete profile and finish it later; the account exists but is excluded from matching until the profile is complete.
Exceptions	**UC1-E1 Invalid credentials.** eduroam rejects the credentials: the system shows an error in the selected language. After three failed attempts it suggests the invitation-code option (UC1-A1). **UC1-E2 eduroam unavailable.** The adapter reports a time-out or error: the system informs the student, offers registration by invitation code or retrying later, and no account is created. **UC1-E3 Account already exists.** The verified e-mail is already registered: the system redirects the student to the login page. **UC1-E4 Invalid invitation code.** The code does not exist, was already used or has expired: the system rejects it and asks for another code. **UC1-E5 Validation error.** Required profile data is missing or invalid: the system highlights the fields and keeps all data already entered.
Postconditions	**Success:** a verified account and a complete profile are stored, the student is logged in and can be matched. **Failure:** no account (or only a draft profile, UC1-A2) is stored.
Patterns	Factory Method (user creation), Adapter (eduroam authentication)

Functional requirements derived from UC1

ID	Requirement
FR-1.1	The system shall allow registration through eduroam verification or a valid invitation code.
FR-1.2	The system shall create the correct user type (Erasmus or local) from the data provided.
FR-1.3	The system shall store skills, hobbies, native languages, languages to practise with level, matching preferences and privacy settings for each profile.
FR-1.4	The system shall reject incomplete profiles from matching until required fields are filled.

ID	Requirement
FR-1.5	The system shall let users edit their profile and delete their account at any time.

3.3 Use Case 2: Find a Match (UC2)

ID / Title	UC2 – Find a Match
Main actor	Student (registered user)
Secondary actors	Candidate student (receives the request); Translation API (through TranslationAdapter)
Goal	Find compatible students according to a chosen criterion and connect with one of them.
Preconditions	1. The student is logged in. 2. The student's profile is complete (UC1).
Trigger	The student opens the *Find a Match* screen.
Main flow	**UC2-S1** The system shows the available matching criteria: Language Exchange, Hobbies and Sports. **UC2-S2** The student selects one criterion and, optionally, filters (language, student type, department, availability). By default Erasmus students are shown local students and vice versa. **UC2-S3** MatchingService sets the matching strategy that corresponds to the selected criterion (LanguageExchangeStrategy, HobbyMatchingStrategy or SportsMatchingStrategy). **UC2-S4** MatchingService loads the candidate profiles through UserRepository, excluding the student, blocked users, users who disabled matching and users with an incomplete profile. **UC2-S5** The selected strategy calculates a compatibility score (0–100) for each candidate. **UC2-S6** The system shows the candidates ordered by score, with the shared interests or complementary languages that explain each score. **UC2-S7** The student opens a candidate's profile and sends a match request. **UC2-S8** The system stores the request as *pending* and notifies the candidate (Receive Notification). **UC2-S9** The candidate accepts the request; the system stores the match as *confirmed* and notifies both students, who can now see each other's contact details.
Alternative flows	**UC2-A1 Change criterion.** At any time after UC2-S6 the student selects another criterion or changes filters; the flow continues at UC2-S3 with the new strategy. **UC2-A2 Translate profile.** At UC2-S7, if the candidate's description is in another language, the student selects *Translate*; the TranslationAdapter returns the text in the interface language. **UC2-A3 Request rejected.** At UC2-S9 the candidate rejects the request; it is stored as *rejected* and the candidate is not shown to the student again for this criterion.
Exceptions	**UC2-E1 Incomplete profile.** The system redirects the student to the profile editor (UC1-S7) with the missing fields highlighted. **UC2-E2 No candidates.** No candidate matches the criterion and filters: the system shows a message and suggests removing filters or trying another criterion. **UC2-E3 Duplicate request.** A pending request to the same candidate already exists: the system informs the student and does not create a new one. **UC2-E4 Translation unavailable.** The translation service fails: the original text is shown with a notice; matching itself is not affected.

Postconditions	**Success:** a ranked list of candidates has been shown; a match request is stored and, if accepted, a confirmed match exists for both students. **Failure:** no match data is changed.
Patterns	Strategy (matching criteria); notifications reuse the Observer mechanism of UC3

Functional requirements derived from UC2

ID	Requirement
FR-2.1	The system shall offer at least three matching criteria: language exchange, hobbies and sports.
FR-2.2	The system shall calculate a compatibility score for each candidate and order results by it.
FR-2.3	The system shall let users filter candidates by language, student type, department and availability.
FR-2.4	The system shall manage match requests with the states pending, confirmed and rejected.
FR-2.5	The system shall never show blocked users or users who disabled matching.

3.4 Use Case 3: Create/Join Event & Get Notified (UC3)

ID / Title	UC3 – Create/Join Event & Get Notified
Main actor	Student (as organiser or as participant)
Secondary actors	Other students subscribed to the event or to its category
Goal	Organise or take part in social, cultural or sports events and be informed of every relevant change.
Preconditions	1. The student is logged in. 2. To join: the event exists, is not cancelled and has not started.
Trigger	The student selects *Create Event* or opens an event from the event list.
Main flow	**UC3-S1** The organiser selects *Create Event* and enters title, description, category (cultural, language, sports, social), date and time, place, capacity and the languages that will be spoken. **UC3-S2** The system validates the data (date in the future, capacity greater than zero, required fields). **UC3-S3** EventService stores the event through EventRepository and registers the organiser as an observer of it. **UC3-S4** EventService notifies the users who follow that category and the organiser's confirmed matches that a new event is available. **UC3-S5** A participant browses or searches events by category, date and language and opens an event. **UC3-S6** The participant selects *Join*. **UC3-S7** EventService checks capacity, adds the participant to the event and registers the participant as an observer of the event. **UC3-S8** The organiser receives a notification about the new participant. **UC3-S9** When the organiser updates or cancels the event, EventService (subject) notifies every registered observer. **UC3-S10** Each user sees the notification in the notification panel, marked as unread until opened.

Alternative flows	**UC3-A1 Leave event.** A participant selects *Leave*; the system removes the participant, detaches them as an observer and notifies the organiser. **UC3-A2 Follow category.** A student subscribes to an event category without joining an event and receives a notification whenever a new event in that category is created (UC3-S4).
Exceptions	**UC3-E1 Invalid event data.** The system highlights the invalid fields and does not store the event. **UC3-E2 Event full.** Capacity has been reached: the *Join* button is disabled and the student can still follow the event to be notified if a place becomes free. **UC3-E3 Already joined.** The system informs the student that they are already a participant. **UC3-E4 Event cancelled or started.** The event cannot be joined; the system shows its status. **UC3-E5 Notification delivery failure.** The notification is stored and shown the next time the user opens the platform; it is never lost.
Postconditions	**Success:** the event and its participant list are stored, and every observer has received the corresponding notification. **Failure:** the event and participant list are unchanged.
Patterns	Observer (EventService as subject, users as observers)

Functional requirements derived from UC3

ID	Requirement
FR-3.1	The system shall allow any registered student to create, edit and cancel their own events.
FR-3.2	The system shall allow students to search events by category, date and language, and to join and leave them while capacity allows.
FR-3.3	The system shall notify all subscribed users when an event is created, updated or cancelled, and the organiser when a participant joins or leaves.
FR-3.4	The system shall notify users about new match requests and confirmed matches (UC2).
FR-3.5	The system shall keep notifications in an in-app panel with read/unread status.

3.5 Use Case 4: Report Inappropriate Content (UC4)

ID / Title	UC4 – Report Inappropriate Content
Main actor	Student (reporter)
Secondary actors	Moderator, Administrator, reported user
Goal	Have inappropriate content (offensive profile, fake account, abusive event) reviewed and handled, so the platform stays safe.
Preconditions	1. The reporter is logged in. 2. The reported content (profile, event or event description) exists and is visible to the reporter.
Trigger	The student selects *Report* on a profile or event.

Main flow	**UC4-S1** The system shows the report form with a list of reasons (spam, harassment, offensive language, fake profile, other) and an optional comment. **UC4-S2** The reporter selects a reason, writes a comment and submits the report. **UC4-S3** ReportService creates a Report with status *Open*, stores it through ReportRepository and passes it to the first handler of the chain. **UC4-S4** AutomaticFilterHandler checks the content against the banned-word list and simple rules. If the violation is clear, it hides the content, sets the report to *Resolved* and the flow continues at UC4-S7. **UC4-S5** Otherwise the report is passed to ModeratorHandler, which places it in the moderators' queue; a moderator reviews it and dismisses it, hides the content or warns the user. **UC4-S6** If the case is serious or repeated, the moderator escalates it and AdminHandler places it in the administrators' queue; an administrator decides (delete content, suspend or ban the account). **UC4-S7** The system records the decision, the handler that resolved the report and the date, and sets the final status. **UC4-S8** The reporter is notified of the outcome, and the reported user is notified of any action taken. The reporter's identity is never shown to the reported user.
Alternative flows	**UC4-A1 Cancel report.** Before UC4-S2 the reporter closes the form; nothing is stored. **UC4-A2 Report dismissed.** At UC4-S5 the moderator finds no violation; the report is closed as *Dismissed* and the content stays visible.
Exceptions	**UC4-E1 Duplicate report.** The same user already reported the same content and the report is still open: the system informs the user and does not create a new report. **UC4-E2 Own content.** Users cannot report their own content; the *Report* option is hidden. **UC4-E3 Missing reason.** The form is not submitted until a reason is selected. **UC4-E4 End of chain.** If no handler resolves the report, it stays in the administrators' queue with status *Pending* and is never discarded.
Postconditions	**Success:** the report is stored with its full handling trail, the content visibility and account status reflect the decision, and the people involved are notified. **Failure:** no report is stored and the content is unchanged.
Patterns	Chain of Responsibility (AutomaticFilter → Moderator → Admin)

Functional requirements derived from UC4

ID	Requirement
FR-4.1	The system shall allow users to report profiles and events with a reason and an optional comment.
FR-4.2	The system shall process each report through automatic filter, moderator and administrator, in this order, until one of them resolves it.
FR-4.3	The system shall provide a moderation panel for moderators and an administration panel for administrators.
FR-4.4	The system shall keep the reporter anonymous to the reported user.
FR-4.5	The system shall record the status history and the resolving handler of every report.

4. Non-Functional Requirements

4.1 Usability

ID	Requirement
NFR-U.1	A new user shall be able to complete registration (UC1) and receive a first list of matches (UC2) in less than 5 minutes.
NFR-U.2	At least 80% of test users (Erasmus and local students) shall rate the platform as easy to use in the end-of-project usability questionnaire.
NFR-U.3	All interface texts, error messages and notifications shall be available in Turkish, English and Spanish, and the user shall be able to switch language at any time without losing entered data.
NFR-U.4	Each core function (profile, matching, events, notifications, report) shall be reachable in at most three clicks from the home page.
NFR-U.5	The interface shall follow basic accessibility rules (WCAG 2.1 AA colour contrast, keyboard navigation, text labels on all form fields).

4.2 Performance

ID	Requirement
NFR-P.1	Pages shall load in under 2 seconds on a standard broadband connection under normal load.
NFR-P.2	Match results (UC2-S3 to UC2-S6) shall be returned in under 3 seconds with up to 1,000 registered users.
NFR-P.3	In-app notifications shall appear within 5 seconds of the triggering action.
NFR-P.4	The automatic filter shall process a report in under 1 second.
NFR-P.5	The system shall support at least 100 concurrent users in the demonstration environment without exceeding the limits above.

4.3 Portability

ID	Requirement
NFR-PO.1	The web interface shall work on the current versions of Chrome, Firefox, Edge and Safari.
NFR-PO.2	The interface shall be responsive, from 360 px wide mobile screens to desktop screens.
NFR-PO.3	The backend shall run on Windows, Linux and macOS without code changes.
NFR-PO.4	Database access shall be limited to the repositories of the Data Access Layer, so a different relational database can be used by changing only that layer.

4.4 Reliability

ID	Requirement
NFR-R.1	All four core use cases shall run end to end without critical errors during the final demo.
NFR-R.2	A failure of an external service (eduroam, translation) shall not stop the platform: the Adapters return a controlled error and the system continues with reduced functionality (UC1-E2, UC2-E4).

ID	Requirement
NFR-R.3	Writes that involve several records (account + profile, event + participant) shall be transactional, so no partial data is stored.
NFR-R.4	Notifications and reports shall be stored persistently and never lost (UC3-E5, UC4-E4).
NFR-R.5	A database backup shall be made before each delivery and before the final demo.

4.5 Security and Privacy

ID	Requirement
NFR-S.1	All communication between browser and server shall use HTTPS.
NFR-S.2	Passwords (invitation-code accounts) shall be stored hashed with a salted algorithm; eduroam passwords shall never be stored.
NFR-S.3	Access shall be role-based: student, moderator and administrator.
NFR-S.4	Only the data needed for matching shall be collected; contact details are visible only after a confirmed match, and users can delete their account and data (GDPR/KVKK principles).

4.6 Maintainability

ID	Requirement
NFR-M.1	Each use case shall be implemented in its own Business Layer service (UserService, MatchingService, EventService, ReportService).
NFR-M.2	A new matching criterion, user type, moderation level or external service shall be added by creating a new class, without modifying existing ones (patterns in Section 7).
NFR-M.3	Class, package and test names shall include the use case identifier they implement (e.g. UC2) to keep traceability.

5. External Interface Requirements

5.1 User Interfaces

Screen	Main interactions	Use case
Landing & language selector	Choose Turkish, English or Spanish; log in or register.	UC1
Registration & profile editor	eduroam or invitation-code sign-up; student type; skills, hobbies, languages, preferences, privacy settings.	UC1
Find a Match	Criterion selector, filters, ranked candidate list with score and shared interests.	UC2
Profile detail	View candidate profile, translate description, send match request, report profile.	UC2, UC4
Events list & detail	Search and filter events, join/leave, follow category, report event.	UC3, UC4
Create / edit event	Form with title, description, category, date, place, capacity, languages.	UC3

Screen	Main interactions	Use case
Notification panel	List of notifications with read/unread status; accept or reject match requests.	UC2, UC3
Report dialog	Reason list and optional comment.	UC4
Moderation & admin panels	Report queues, content preview, decisions, escalation, account suspension, invitation-code management.	UC4, UC1

5.2 Software Interfaces

Interface	Description
eduroam authentication	Reached through the *AuthenticationProvider* interface implemented by *EduroamAdapter*. Input: institutional credentials; output: verified identity or error. A mock implementation is used in this project.
Translation API	Reached through the *TranslationService* interface implemented by *TranslationAdapter*. Input: text, source and target language; output: translated text. A mock implementation is used in this project.
Relational database	Accessed only by UserRepository, EventRepository and ReportRepository in the Data Access Layer.
Web UI – backend	The Presentation Layer communicates with the Business Layer through an HTTP/JSON (REST-style) interface over HTTPS.

6. Software Architecture Overview

Section author: Alberto Alvarez Polo – integrated by Carlos Molina Perez.

6.1 Architectural Style: Layered Architecture

The platform uses a Layered Architecture with three internal layers (Presentation, Business and Data Access) and a set of external services connected through Adapters. Each layer only communicates with the layer directly below it. We chose this style for three reasons:

- **Separation of concerns:** user interface, business rules and data storage are kept apart, so a change in one layer does not break the others.
- **Maintainability and testability:** each service can be developed and tested independently, which suits a three-person team working in parallel.
- **Natural fit for design patterns:** the Business Layer is where Factory Method, Strategy, Observer and Chain of Responsibility live, and Adapters isolate the system from external services such as eduroam and the translation API.

6.2 Architecture Diagram

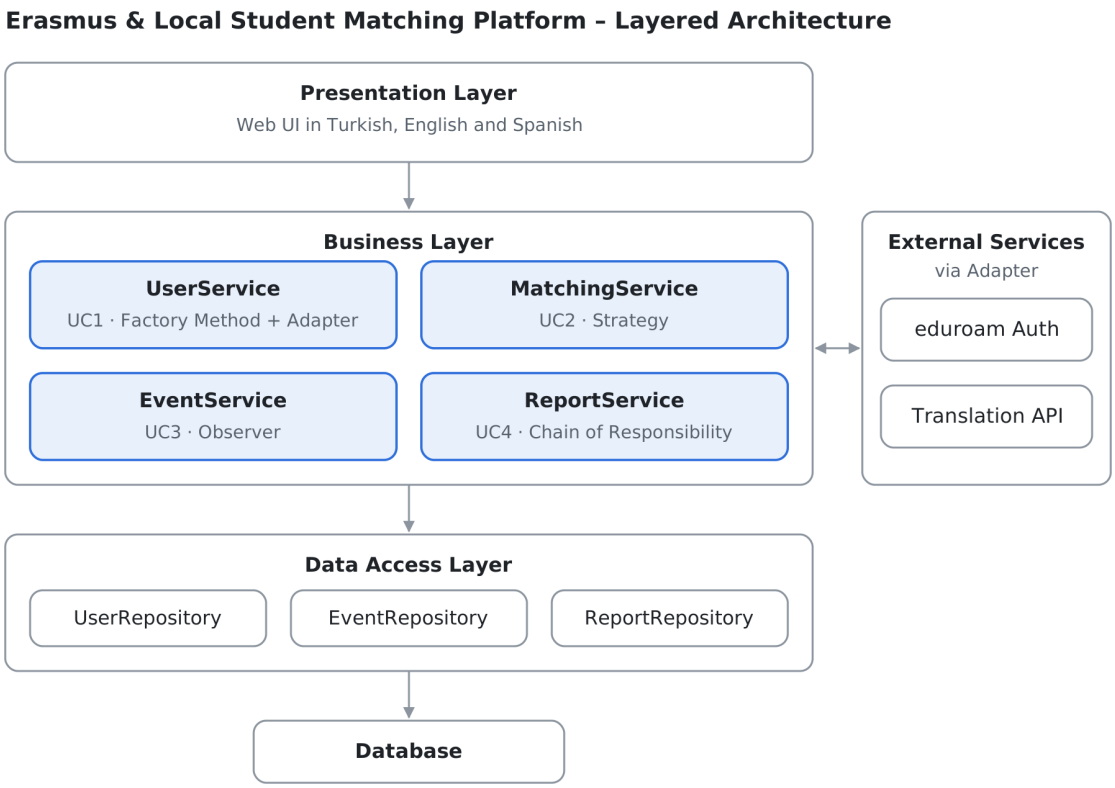


Figure 2 – Layered architecture: requests flow from the Web UI down to the Business Layer services and from there to the repositories and the database; external services are reached only through Adapters.

6.3 Layer Descriptions

Layer	Components	Responsibility
Presentation	Web UI (Turkish / English / Spanish)	Displays screens, collects user input and shows results and notifications
Business	UserService, MatchingService, EventService, ReportService	Implements the business rules of each use case and applies the design patterns
Data Access	UserRepository, EventRepository, ReportRepository, Database	Stores and retrieves users, profiles, events, matches and reports
External Services	eduroam Authentication, Translation API (via Adapter)	Third-party systems wrapped by Adapters so the Business Layer does not depend on their interfaces

6.4 How the Architecture Supports the Use Cases

Each use case has its own service in the Business Layer, so the four use cases can be designed, implemented and tested independently.

Use case	Layers involved	Pattern applied
UC1 Register & Create Profile	Web UI → UserService → UserRepository; eduroam Adapter for authentication	Factory Method, Adapter

Use case	Layers involved	Pattern applied
UC2 Find a Match	Web UI → MatchingService → UserRepository; Translation Adapter for profile texts	Strategy
UC3 Create/Join Event & Get Notified	Web UI → EventService → EventRepository; notifications sent back to subscribed users	Observer
UC4 Report Inappropriate Content	Web UI → ReportService → ReportRepository	Chain of Responsibility

- **UC1:** the UserService uses a Factory Method to create the right user type (Erasmus or local) and an Adapter to verify the user through eduroam without depending on its interface.
- **UC2:** the MatchingService delegates the matching logic to an interchangeable Strategy (language exchange, hobbies, sports), so new criteria can be added without changing the service.
- **UC3:** the EventService acts as the subject of an Observer, notifying every subscribed user when a match or event changes.
- **UC4:** the ReportService passes each report along a chain of handlers (automatic filter → moderator → admin) until one of them resolves it.

7. Pattern–Use Case Mapping

Section author: Alejandro Blas Piris – integrated by Carlos Molina Perez.

The following matrix defines the design patterns selected for the platform's core use cases. Each pattern is assigned to solve a specific architectural challenge, maximising the flexibility, modularity and maintainability of the Business Layer in our Layered Architecture.

7.1 Mapping Matrix and Justification

ID	Use case	Pattern	Application and architectural justification
UC1	Register & Create Profile	Factory Method + Adapter	**Factory Method** is applied in the UserService to centralise the creation logic for different user profiles (Erasmus vs. local students). This ensures modularity, as new profile types can be added without modifying existing instantiation logic. **Adapter** is used to wrap external services, specifically the eduroam authentication mechanism. It isolates our Business Layer from external API changes, enhancing maintainability by allowing us to simulate the service with a mock during development.
UC2	Find a Match	Strategy	The Strategy pattern provides runtime flexibility to the MatchingService. Depending on user preferences, the algorithm used to find compatible students needs to change (e.g. matching by language exchange goals, by hobbies, or by sports). Strategy allows these criteria to be encapsulated into interchangeable classes, so new filters can be introduced without altering the core matching service.
UC3	Create/Join Event & Get Notified	Observer	The Observer pattern fits the notification requirement. The EventService acts as the Subject, while subscribed users act as Observers. When an event is updated or a new match is found, the system automatically notifies all interested users. This decouples event management logic from notification delivery, increasing the overall modularity of the system.

ID	Use case	Pattern	Application and architectural justification
UC4	Report Inappropriate Content	Chain of Responsibility	Chain of Responsibility organises the moderation workflow into a sequence of handlers (Automatic Filter → Moderator → System Admin). When a report is submitted via ReportService, it passes along this chain until a handler can resolve it. Escalation rules or new moderation levels can be added without rewriting the reporting service.

7.2 Pattern Participants in the Architecture

The table below names the classes that play each role in the pattern, so the mapping can be traced to the design (PA2/PA3) and to the source code.

Use case	Pattern	Role → class	Steps
UC1	Factory Method	Creator: *UserCreator* (abstract, *createUser()*) Concrete creators: *ErasmusStudentCreator*, *LocalStudentCreator* Product: *Student* → *ErasmusStudent*, *LocalStudent*	UC1-S6
UC1, UC2	Adapter	Target: *AuthenticationProvider*, *TranslationService* Adapters: *EduroamAdapter*, *TranslationAdapter* Adaptees: *EduroamClient*, *TranslationApiClient* (mocks)	UC1-S4, UC2-A2
UC2	Strategy	Context: *MatchingService* Strategy: *MatchingStrategy* (*calculateScore()*) Concrete: *LanguageExchangeStrategy*, *HobbyMatchingStrategy*, *SportsMatchingStrategy*	UC2-S3, UC2-S5
UC3	Observer	Subject: *EventService* (*attach()*, *detach()*, *notifyObservers()*) Observer: *EventObserver* (*update()*) Concrete observer: *UserNotificationObserver*	UC3-S3, S4, S7–S9
UC4	Chain of Responsibility	Handler: *ReportHandler* (*setNext()*, *handle()*) Concrete handlers: *AutomaticFilterHandler* → *ModeratorHandler* → *AdminHandler* Client: *ReportService*	UC4-S3 to UC4-S6

8. Traceability Matrix

The matrix links each use case to its requirements, its components, its patterns and the acceptance tests that will verify it. The acceptance tests refine the acceptance criteria of the Project Plan (Section 10) and will be detailed in the test documentation.

Use case	Requirements	Components	Patterns	Acceptance tests
UC1	PF-1, PF-6; FR-1.1–1.5; NFR-U.1, NFR-R.2, NFR-R.3, NFR-S.2	Web UI, UserService, UserRepository, EduroamAdapter	Factory Method, Adapter	AT-UC1-01 eduroam registration AT-UC1-02 invitation code AT-UC1-03 eduroam unavailable

Use case	Requirements	Components	Patterns	Acceptance tests
UC2	PF-2, PF-4; FR-2.1–2.5; NFR-U.1, NFR-P.2	Web UI, MatchingService, UserRepository, TranslationAdapter	Strategy (Adapter)	AT-UC2-01 match per criterion AT-UC2-02 request accepted AT-UC2-03 no candidates
UC3	PF-3, PF-4; FR-3.1–3.5; NFR-P.3, NFR-R.4	Web UI, EventService, EventRepository	Observer	AT-UC3-01 create event AT-UC3-02 join full event AT-UC3-03 update notifies all
UC4	PF-5; FR-4.1–4.5; NFR-P.4, NFR-R.4, NFR-S.3	Web UI, ReportService, ReportRepository	Chain of Responsibility	AT-UC4-01 auto-filter resolves AT-UC4-02 moderator resolves AT-UC4-03 escalated to admin

9. Task Matrix

Task	Responsible
Introduction, product perspective and product functions	Carlos Molina Perez
User characteristics, constraints and assumptions	Carlos Molina Perez
UML use case diagram	Carlos Molina Perez
Use case descriptions UC1–UC4 (flows, exceptions, functional requirements)	Carlos Molina Perez
Non-functional requirements	Carlos Molina Perez
External interface requirements	Carlos Molina Perez
Software Architecture Overview (Section 6)	Alberto Alvarez Polo
Pattern–Use Case Mapping Matrix (Section 7.1)	Alejandro Blas Piris
Pattern participants and traceability matrix (Sections 7.2, 8)	Carlos Molina Perez
Review of use cases against the architecture	Alberto Alvarez Polo
Review of use cases against the pattern mapping	Alejandro Blas Piris
Integration of all sections and final formatting	Carlos Molina Perez